



INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	SW-AyLP-APE-06: Arreglos y Listas
Unidad de Organización Curricular:	PROFESIONAL
Nivel y Paralelo:	1 Semestre Ingeniería en Software “B”
Alumnos participantes:	Guaman Chanahuano Hamilton Alexander
Asignatura:	Algoritmos y Lógica de Programación
Docente:	Ing. JOSE RUBEN CAIZA CAIZABUANO
Git:	

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

Conocer en manejo de arreglos y listas que permiten la manipulación de grandes cantidades de datos

Específicos:

- Crear y manipular ArrayList/vector con colecciones de nombres y notas de estudiantes aplicando métodos de agregar y recorrer elementos.
- Desarrollar un sistema de inventario y menú CRUD funcional utilizando ArrayList en Java y vector en C++ para gestionar datos dinámicamente.
- Comparar el comportamiento y rendimiento entre arreglos estáticos y ArrayList/vector al realizar inserciones masivas de datos.
- Implementar LinkedList/list para simular estructuras de cola (FIFO) en sistemas de turnos y filas de atención al cliente.
- Gestionar el control de versiones del código fuente mediante un repositorio estructurado en GitHub.

2.2 Modalidad

Presencial

2.3 Tiempo de duración

Presenciales: 6

No presenciales: 0

2.4 Instrucciones

- La tarea se debe realizar de forma individual. - Lea detenidamente cada uno de los ejercicios planteados en el aula virtual y desarrolle lo solicitado. - Codifique en Java cada uno de los ejercicios planteados. - Al finalizar, comprimir y subir al aula virtual el proyecto desarrollado, adicionalmente realizar capturas de cada una de las clases y métodos de los ejercicios. Las capturas se subirán en un archivo pdf en el formato de Guías APE

2.5 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

- Computadora personal.



- Entorno de Desarrollo Integrado (IDE) para C++ (ej. Code::Blocks o VS Code).
- Entorno de Desarrollo o JDK para la compilación en Java.

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

- ☒ Plataformas educativas
- ☐ Simuladores y laboratorios virtuales
- ☒ Aplicaciones educativas
- ☐ Recursos audiovisuales
- ☐ Gamificación
- ☒ Inteligencia Artificial

Otros (Especifique): _

2.6 Actividades por desarrollar

Tabla de Equivalencias Java ↔ C++

La siguiente tabla resume las equivalencias entre las estructuras de datos dinámicas utilizadas en los ejercicios:

Java	C++	Descripción
ArrayList	<code>vector<T></code>	Lista dinámica de elementos
LinkedList	<code>list<T></code>	Lista enlazada bidireccional
add()	<code>push_back()</code>	Agrega elemento al final
remove(i)	<code>erase(begin()+i)</code>	Elimina elemento en posición i
get(i)	<code>[i]</code>	Accede al elemento en posición i
size()	<code>size()</code>	Retorna la cantidad de elementos
isEmpty()	<code>empty()</code>	Verifica si la lista está vacía

Ejercicio 1 – ArrayList I – Lista de Nombres

Enunciado del Problema

Crear un ArrayList en Java (vector en C++) con 5 nombres de estudiantes. El programa debe solicitar al usuario el ingreso de cada nombre y mostrar la lista completa al finalizar.

Análisis del Problema

El problema requiere el uso de una estructura de datos dinámica para almacenar cadenas de texto. Se necesita un ciclo que se repita 5 veces para registrar los nombres, y posteriormente otro ciclo para recorrer e imprimir cada elemento. La diferencia principal entre el arreglo estático y el ArrayList radica en que el segundo puede crecer dinámicamente sin necesidad de definir su tamaño de antemano.

Variables de Entrada y Salida

Entrada

- lista[5]: arreglo de 5 espacios para almacenar texto (Caracter).
- nombre: variable temporal de tipo texto para leer la entrada del usuario.

Proceso

- Recorrer un ciclo de 1 hasta 5.
- Solicitar al usuario que ingrese un nombre en cada iteración.
- Almacenar el nombre ingresado directamente en la posición i del arreglo lista.

Salida

- Impresión en pantalla de la lista secuencial mostrando el índice y el nombre de los 5 estudiantes registrados



Variable	Tipo de Dato	Descripción	E / S
nombre	String / string	Nombre del estudiante	Entrada
I	Int	Variable de control del ciclo	Proceso
estudiantes	ArrayList<String> / vector<string>	Colección de nombres	Proceso
Listado impreso	String	Nombres numerados en pantalla	Salida

Algoritmo

1. Inicio del programa
2. Declarar una lista dinámica (ArrayList/vector) de tipo String.
3. Iniciar un ciclo que se repita 5 veces.
4. En cada iteración, solicitar y leer el nombre del estudiante.
5. Agregar el nombre ingresado a la lista con add()/push_back().
6. Finalizar el ciclo de ingreso.
7. Recorrer la lista e imprimir cada nombre con su número de posición.
8. Fin del programa.

Pseudocódigo (PSeInt)

Algoritmo ListaEstudiantes

Definir nombre, lista Como Caracter

Definir i Como Entero

Dimension lista[5]

Para i <- 1 Hasta 5 Hacer

 Escribir Sin Saltar "Nombre del estudiante ", i, ": "

 Leer nombre

 lista[i] <- nombre

FinPara

Escribir "--- Lista de Estudiantes ---"

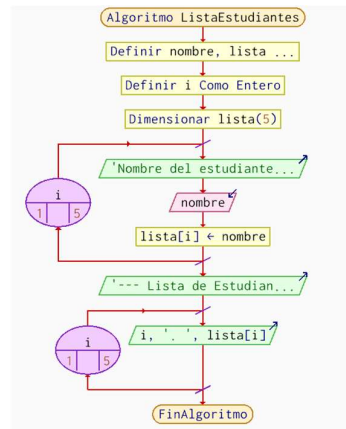
Para i <- 1 Hasta 5 Hacer

 Escribir i, ". ", lista[i]

FinPara

FinAlgoritmo

Diagrama de Flujo



Código C++

```
#include <iostream>
#include <vector>
#include <string>
using namespace std;

int main() {
    // Crear un vector con 5 nombres de estudiantes
    vector<string> estudiantes;
    string nombre;

    cout << "=== REGISTRO DE 5 ESTUDIANTES ===" << endl;
    for (int i = 0; i < 5; i++) {
        cout << "Ingrese nombre del estudiante " << i+1 << ": ";
        cin >> nombre;
        estudiantes.push_back(nombre); // Agregar al vector
    }

    // Mostrar la lista completa
    cout << "\n--- Lista de Estudiantes ---" << endl;
    for (int i = 0; i < estudiantes.size(); i++) {
        cout << i+1 << ". " << estudiantes[i] << endl;
    }
    return 0;
}
```

Prueba de escritorio Evidencia

<pre>=== REGISTRO DE 5 ESTUDIANTES === Ingrese nombre del estudiante 1: Alexander Ingrese nombre del estudiante 2: Ian Ingrese nombre del estudiante 3: Miguel Ingrese nombre del estudiante 4: Domenica Ingrese nombre del estudiante 5: Karla --- Lista de Estudiantes --- 1. Alexander 2. Ian 3. Miguel 4. Domenica 5. Karla PS C:\Users\ready\OneDrive\Documents\PROGRAMACION_UTA\practicas></pre>	<pre>=== REGISTRO DE 5 ESTUDIANTES === Ingrese nombre del estudiante 1: sOLIS Ingrese nombre del estudiante 2: Vela Ingrese nombre del estudiante 3: Karen Ingrese nombre del estudiante 4: Mikaela Ingrese nombre del estudiante 5: Maria --- Lista de Estudiantes --- 1. sOLIS 2. Vela 3. Karen 4. Mikaela 5. Maria PS C:\Users\ready\OneDrive\Documents\PROGRAMACION_UTA\practicas></pre>
--	--

Imagen 1 y 2 Evidencia C++ hecha en VisualCode



Código Java

```
import java.util.ArrayList;
import java.util.Scanner;

public class Ejercicio1 {
    public static void main(String[] args) {
        // Crear un ArrayList para almacenar nombres
        ArrayList<String> estudiantes = new ArrayList<>();
        Scanner sc = new Scanner(System.in);

        System.out.println("=== REGISTRO DE 5 ESTUDIANTES ===");
        for (int i = 0; i < 5; i++) {
            System.out.print("Ingrese nombre del estudiante " + (i+1) + ": ");
            String nombre = sc.next();
            estudiantes.add(nombre); // Agregar al ArrayList
        }

        // Mostrar la lista completa
        System.out.println("\n--- Lista de Estudiantes ---");
        for (int i = 0; i < estudiantes.size(); i++) {
            System.out.println((i+1) + ". " + estudiantes.get(i));
        }
        sc.close();
    }
}
```

Prueba de escritorio Evidencia

```
=== REGISTRO DE 5 ESTUDIANTES (JAVA) ===
Ingrese nombre del estudiante 1: Miguel
Ingrese nombre del estudiante 2: Alex
Ingrese nombre del estudiante 3: Gina
Ingrese nombre del estudiante 4: Hanna
Ingrese nombre del estudiante 5: Selene

___Lista de Estudiantes___
1. Miguel
2. Alex
3. Gina
4. Hanna
5. Selene
PS C:\Users\ready\OneDrive\Documents\PROGRAMACION_UTA\practicas>
```

```
dhat.java\jdt_ws\practicas_d5ab0d49\bin\ 'Ejercicio1'
== REGISTRO DE 5 ESTUDIANTES (JAVA) ==
ngrese nombre del estudiante 1: Ian
ngrese nombre del estudiante 2: Jhair
ngrese nombre del estudiante 3: Domenica
ngrese nombre del estudiante 4: Estefania
ngrese nombre del estudiante 5: Maria

___Lista de Estudiantes___
. Ian
. Jhair
. Domenica
. Estefania
. Maria
S C:\Users\ready\OneDrive\Documents\PROGRAMACION_UTA\practicas>
```

Imagen 3 y 4 Evidencia Java hecha en VisualCode

Pruebas de Escritorio

Caso	Entrada	Proceso	Salida Esperada
1	Ana, Luis, María, Pedro, Sofia	5 add() al ArrayList/vector	1.Ana 2.Luis 3.María 4.Pedro 5.Sofia
2	Carlos, Juan, Eva, Rox, Tom	5 push_back() al vector	1.Carlos 2.Juan 3.Eva 4.Rox 5.Tom
3	Z, A, M, B, W	Nombres de 1 letra – válidos	1.Z 2.A 3.M 4.B 5.W



Ejercicio 2 – ArrayList I – Registro de Notas

Enunciado del Problema

Crear un ArrayList/vector con 6 notas numéricas (tipo double/float). El programa debe registrar las 6 notas ingresadas por el usuario, mostrarlas individualmente y calcular el promedio general del grupo.

Análisis del Problema

Se requiere una colección dinámica de valores numéricos de punto flotante. El proceso incluye la acumulación de la suma para calcular el promedio final. Se aplica un ciclo de 6 iteraciones para el ingreso y otro para la presentación de resultados.

Variables de Entrada y Salida

Entrada

- notas[6]: arreglo de 6 espacios para almacenar números decimales (Real).
- nota: variable numérica ingresada por el usuario.

Proceso

- Inicializar una variable suma en 0.
- Recorrer un ciclo 6 veces para solicitar la calificación.
- Guardar cada nota en el arreglo y acumular su valor sumándolo a la variable suma.
- Calcular el promedio dividiendo la suma total para 6.

Salida

- El valor final calculado y almacenado en la variable promedio.

Variable	Tipo de Dato	Descripción	E / S
nota	double / float	Nota individual ingresada	Entrada
notas	ArrayList<Double> / vector<float>	Colección de notas	Proceso
suma	double / float	Acumulador de la suma total	Proceso
promedio	double / float	Promedio calculado de las 6 notas	Salida

Algoritmo

1. Inicio del programa.
2. Declarar lista dinámica y variable acumuladora suma = 0.
3. Ciclo 6 veces: solicitar nota, agregarla a la lista, sumar a acumulador.
4. Calcular promedio = suma / 6.
5. Recorrer la lista mostrando cada nota.
6. Mostrar el promedio general.
7. Fin del programa.

Pseudocódigo (PSeInt)

Algoritmo RegistroNotas

Definir notas, nota, suma, promedio Como Real

Definir i Como Entero

Dimension notas[6]

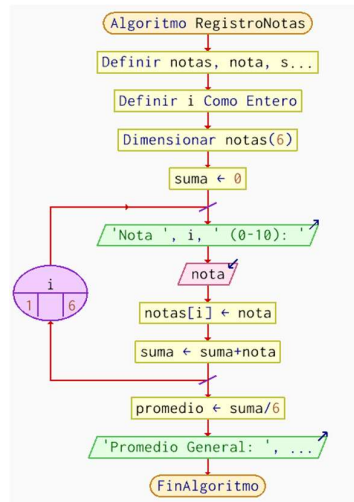
suma <- 0

Para i <- 1 Hasta 6 Hacer



```
Escribir Sin Saltar "Nota ", i, " (0-10): "  
Leer nota  
notas[i] <- nota  
suma <- suma + nota  
FinPara  
  
promedio <- suma / 6  
Escribir "Promedio General: ", promedio  
FinAlgoritmo
```

Diagrama de Flujo



Código C++

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<float> notas;
    float nota;
    float suma = 0;

    cout << "=== REGISTRO DE 6 NOTAS ===" << endl;
    for (int i = 0; i < 6; i++) {
        cout << "Ingrese nota " << i+1 << " (0-10): ";
        cin >> nota;
        notas.push_back(nota); // Agregar nota al vector
        suma += nota;
    }

    // Calcular el promedio
    float promedio = suma / 6;

    cout << "\n--- Notas Registradas ---" << endl;
    for (int i = 0; i < notas.size(); i++) {
        cout << "Nota " << i+1 << ": " << notas[i] << endl;
    }
    cout << "Promedio General: " << promedio << endl;
```



```
return 0;  
}
```

Prueba de Escritorio

```
t64\bin\gdb.exe' '--interpreter=mi'  
=== REGISTRO DE 6 NOTAS ===  
Ingrese nota 1 (0-10): 10  
Ingrese nota 2 (0-10): 9  
Ingrese nota 3 (0-10): 9  
Ingrese nota 4 (0-10): 10  
Ingrese nota 5 (0-10): 9  
Ingrese nota 6 (0-10): 10  
--- Notas Registradas ---  
Nota 1: 10  
Nota 2: 9  
Nota 3: 9  
Nota 4: 10  
Nota 5: 9  
Nota 6: 10  
Promedio General: 9.5  
PS C:\Users\ready\OneDrive\Documents\PROGRAMACION_UTA\practicas>  
  
t64\bin\gdb.exe' '--interpreter=mi'  
=== REGISTRO DE 6 NOTAS ===  
Ingrese nota 1 (0-10): 10  
Ingrese nota 2 (0-10): 10  
Ingrese nota 3 (0-10): 10  
Ingrese nota 4 (0-10): 10  
Ingrese nota 5 (0-10): 9  
Ingrese nota 6 (0-10): 9  
--- Notas Registradas ---  
Nota 1: 10  
Nota 2: 10  
Nota 3: 10  
Nota 4: 10  
Nota 5: 9  
Nota 6: 9  
Promedio General: 9.66667  
PS C:\Users\ready\OneDrive\Documents\PROGRAMACION_UTA\practicas>
```

Imagen 5 y 6 Evidencia C++ hecha en VisualCode

Código Java

```
import java.util.ArrayList;  
import java.util.Scanner;  
  
public class Ejercicio2 {  
    public static void main(String[] args) {  
        ArrayList<Double> notas = new ArrayList<>();  
        Scanner sc = new Scanner(System.in);  
        double suma = 0;  
  
        System.out.println("=== REGISTRO DE 6 NOTAS ===");  
        for (int i = 0; i < 6; i++) {  
            System.out.print("Ingrese nota " + (i+1) + " (0-10): ");  
            double nota = sc.nextDouble();  
            notas.add(nota); // Agregar al ArrayList  
            suma += nota;  
        }  
  
        double promedio = suma / 6;  
        System.out.println("\n--- Notas Registradas ---");  
        for (int i = 0; i < notas.size(); i++) {  
            System.out.println("Nota " + (i+1) + ": " + notas.get(i));  
        }  
        System.out.printf("Promedio General: %.2f\n", promedio);  
        sc.close();  
    }  
}
```

Prueba de Escritorio



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```

63541256\redhat_java\jdk\src\practicas_d5ab0d49\bin\ 'Ejercicio2'
=== REGISTRO DE 6 NOTAS ===
Ingrese nota 1 (0-10): 9
Ingrese nota 2 (0-10): 9
Ingrese nota 3 (0-10): 9
Ingrese nota 4 (0-10): 9
Ingrese nota 5 (0-10): 9
Ingrese nota 6 (0-10): 10

--- Notas Registradas ---
Nota 1: 9.0
Nota 2: 9.0
Nota 3: 9.0
Nota 4: 9.0
Nota 5: 9.0
Nota 6: 10.0
Promedio General: 9.17
PS C:\Users\ready\OneDrive\Documents\PROGRAMACION_UTA\practicas>

63541256\redhat_java\jdk\src\practicas_d5ab0d49\bin\ 'Ejercicio2'
=== REGISTRO DE 6 NOTAS ===
Ingrese nota 1 (0-10): 8
Ingrese nota 2 (0-10): 8
Ingrese nota 3 (0-10): 8
Ingrese nota 4 (0-10): 8
Ingrese nota 5 (0-10): 9
Ingrese nota 6 (0-10): 9

--- Notas Registradas ---
Nota 1: 8.0
Nota 2: 8.0
Nota 3: 8.0
Nota 4: 8.0
Nota 5: 9.0
Nota 6: 9.0
Promedio General: 8.33
PS C:\Users\ready\OneDrive\Documents\PROGRAMACION_UTA\practicas>

```

Imagen 7 y 8 Evidencia Java hecha en VisualCode

Pruebas de Escritorio

Caso	Entrada	Proceso	Salida Esperada
1	8,9,7,10,6,8	suma=48; prom=48/6	Promedio: 8.00
2	5,5,5,5,5,5	suma=30; prom=30/6	Promedio: 5.00
3	10,10,10,10,10,10	suma=60; prom=60/6	Promedio: 10.00

Ejercicio 3 – ArrayList I – Registro y Búsqueda de Productos

Enunciado del Problema

Registrar una cantidad variable de productos en una lista dinámica (ArrayList/vector). Una vez registrados, el programa permite al usuario buscar un producto por su nombre y responde indicando si fue encontrado y en qué posición.

Análisis del Problema

Se utiliza la función indexOf() en Java y find() de <algorithm> en C++ para la búsqueda lineal. El programa debe gestionar el caso en que el producto no exista en la lista. El usuario decide cuántos productos registrar (n variable).

Variables de Entrada y Salida

Entrada

- n: cantidad total de productos a registrar (Entero).
- productos[n]: arreglo dimensionado dinámicamente según el valor de n.
- buscar: cadena de texto con el nombre del producto objetivo a localizar.

Proceso

- Recorrer un ciclo n veces para poblar el arreglo con los nombres de los productos.
- Leer el valor de la variable buscar.
- Recorrer secuencialmente el arreglo comparando si el valor en productos[i] es exactamente igual a buscar.
- Detectar si hay coincidencia y guardar la posición actual i en la variable idx.

Salida

- Mensaje indicando "Encontrado en posicion idx" si el producto existe.
- Mensaje "No encontrado" si la variable idx no registró coincidencias.

Variable	Tipo de Dato	Descripción	E / S
N	Int	Cantidad de productos a registrar	Entrada
Product	String / string	Nombre de cada product	Entrada



Buscar	String / string	Nombre del producto a buscar	Entrada
Idx	Int	Posición encontrada (-1 si no existe)	Salida

Algoritmo

1. Inicio del programa.
2. Solicitar la cantidad n de productos.
3. Ciclo n veces: leer nombre del producto y agregarlo a la lista.
4. Solicitar el nombre del producto a buscar.
5. Buscar el producto en la lista (indexOf / find).
6. Si se encuentra, mostrar su posición; de lo contrario, mostrar mensaje de no encontrado.
7. Fin del programa.

Pseudocódigo (PSeInt)

Algoritmo BuscarProducto

Definir productos, producto, buscar Como Caracter

Definir n, i, idx Como Entero

Escribir Sin Saltar "¿Cuántos productos vas a ingresar? "

Leer n

Dimension productos[n]

Para i <- 1 Hasta n Hacer

 Escribir Sin Saltar "Producto ", i, ": "

 Leer producto

 productos[i] <- producto

FinPara

Escribir Sin Saltar "Buscar producto: "

Leer buscar

idx <- -1

Para i <- 1 Hasta n Hacer

 Si productos[i] = buscar Entonces

 idx <- i

 FinSi

FinPara

Si idx >= 1 Entonces

 Escribir "Encontrado en posicion ", idx

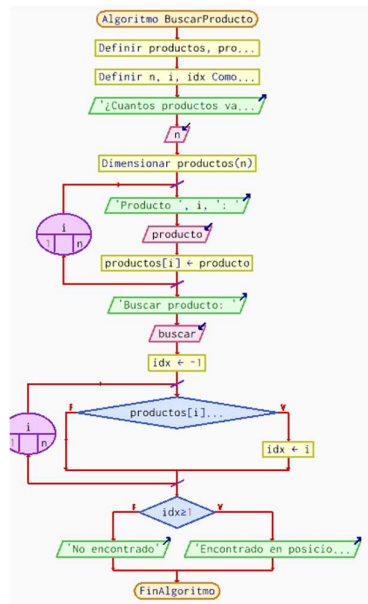
SiNo

 Escribir "No encontrado"

FinSi

FinAlgoritmo

Diagrama de Flujo



Código C++

```
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>
using namespace std;

int main() {
    vector<string> productos;
    string producto, buscar;
    int n;

    cout << "¿Cuántos productos desea registrar? ";
    cin >> n;

    // Registrar productos
    for (int i = 0; i < n; i++) {
        cout << "Producto " << i+1 << ": ";
        cin >> producto;
        productos.push_back(producto);
    }

    // Buscar producto
    cout << "\nIngrese el producto a buscar: ";
    cin >> buscar;

    auto it = find(productos.begin(), productos.end(), buscar);
    if (it != productos.end()) {
        cout << "Producto ENCONTRADO en la posicion: "
            << distance(productos.begin(), it) << endl;
    } else {
        cout << "Producto NO encontrado." << endl;
    }
    return 0;
}
```



Prueba de Escritorio

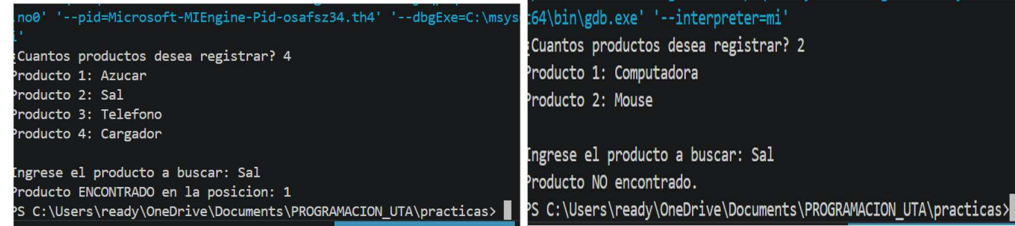


Imagen 9 y 10 Evidencia C++ hecha en VisualCode

Código Java

```
import java.util.ArrayList;
import java.util.Scanner;

public class Ejercicio3 {
    public static void main(String[] args) {
        ArrayList<String> productos = new ArrayList<>();
        Scanner sc = new Scanner(System.in);

        System.out.print("¿Cuantos productos desea registrar? ");
        int n = sc.nextInt();

        // Registrar productos
        for (int i = 0; i < n; i++) {
            System.out.print("Producto " + (i+1) + ": ");
            productos.add(sc.next());
        }

        // Buscar producto
        System.out.print("\nIngrese el producto a buscar: ");
        String buscar = sc.next();

        int idx = productos.indexOf(buscar);
        if (idx != -1) {
            System.out.println("Producto ENCONTRADO en posicion: " + idx);
        } else {
            System.out.println("Producto NO encontrado.");
        }
        sc.close();
    }
}
```

Prueba de Escritorio

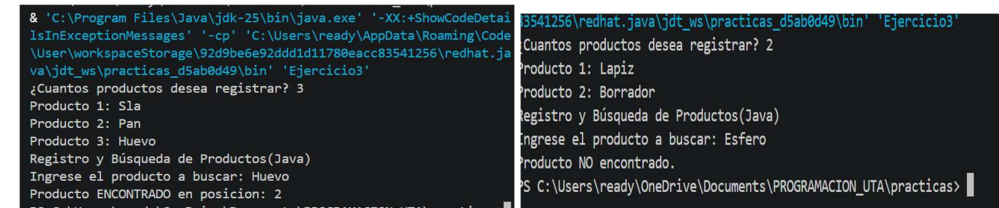


Imagen 11 y 12 Evidencia Java hecha en VisualCode

Pruebas de Escritorio



Caso	Entrada	Proceso	Salida Esperada
1	n=3: Leche,Pan,Queso; buscar=Pan	indexOf('Pan')=1	Encontrado en posición: 1
2	n=3: A,B,C; buscar=D	indexOf('D')=-1	Producto NO encontrado.
3	n=4: Agua,Sal,Arroz,Azucar; buscar=Arroz	indexOf('Arroz')=2	Encontrado en posición: 2

Ejercicio 4 – ArrayList II – Menú CRUD

Enunciado del Problema

Desarrollar un menú interactivo CRUD (Crear, Leer, Actualizar, Eliminar) utilizando un ArrayList en Java y un vector en C++. El usuario puede agregar elementos, visualizarlos, modificar un elemento existente por su posición, o eliminarlo.

Análisis del Problema

Se requiere un menú en bucle do-while que permita al usuario ejecutar las cuatro operaciones básicas sobre la lista. Las operaciones de actualización (set/[pos]=) y eliminación (remove/erase) requieren validación de la posición ingresada para evitar accesos fuera de rango.

Variables de Entrada y Salida

Entrada

- datos[100]: arreglo de texto con un límite fijo alto para simular almacenamiento dinámico.
- op: valor entero que representa la opción elegida del menú.
- item: cadena de texto a guardar o actualizar en el inventario.
- pos: número entero que indica la posición a modificar o eliminar.

Proceso

- Mostrar un menú interactivo dentro de un ciclo Repetir.
- **Agregar (1):** Incrementar el contador global n y guardar el item en la posición nueva.
- **Actualizar (3):** Validar que pos esté dentro del rango (1 a n) y sobrescribir el dato en datos[pos].
- **Eliminar (4):** Detectar la posición a borrar y ejecutar un ciclo para desplazar todos los elementos un espacio hacia arriba (datos[i] <- datos[i+1]), reduciendo finalmente el tamaño lógico n en 1.

Salida

- Lista de elementos actualizados mostrados al usuario cuando selecciona la opción "Ver (2)".
- Mensajes de alerta ("Posicion invalida") si el usuario intenta alterar un índice que no existe.

Variable	Tipo de Dato	Descripción	E / S
Opcion	int	Opción del menú seleccionada	Entrada
Item	String / string	Elemento a agregar o actualizar	Entrada
Pos	int	Posición para actualizar o eliminar	Entrada



Datos	ArrayList/vector	Lista de elementos gestionados	Proceso/Salida
-------	------------------	--------------------------------	----------------

Algoritmo

1. Inicio del programa.
2. Declarar la lista dinámica vacía.
3. Iniciar ciclo do-while.
4. Mostrar el menú con las opciones: 1-Agregar, 2-Mostrar, 3-Actualizar, 4-Eliminar, 0-Salir.
5. Leer la opción del usuario.
6. Si opción=1: solicitar elemento y agregar a la lista.
7. Si opción=2: recorrer e imprimir todos los elementos.
8. Si opción=3: mostrar lista, pedir posición y nuevo valor, validar y actualizar.
9. Si opción=4: mostrar lista, pedir posición, validar y eliminar.
10. Repetir mientras opción != 0.
11. Fin del programa.

Pseudocódigo (PSeInt)

Algoritmo MenuCRUD

Definir datos, item Como Caracter

Definir op, pos, n, i Como Entero

Dimension datos[100]

n <- 0

Repetir

Escribir "1.Agregar 2.Ver 3.Actualizar 4.Eliminar 0.Salir"

Escribir Sin Saltar "Opcion: "

Leer op

Segun op Hacer

1:

Escribir Sin Saltar "Item a agregar: "

Leer item

n <- n + 1

datos[n] <- item

2:

Para i <- 1 Hasta n Hacer

Escribir i, ". ", datos[i]

FinPara

3:

Escribir Sin Saltar "Posicion a actualizar (1-", n, "): "

Leer pos

Si pos >= 1 Y pos <= n Entonces

Escribir Sin Saltar "Nuevo item: "

Leer item

datos[pos] <- item

FinSi

4:

Escribir Sin Saltar "Posicion a eliminar (1-", n, "): "

Leer pos

Si pos >= 1 Y pos <= n Entonces

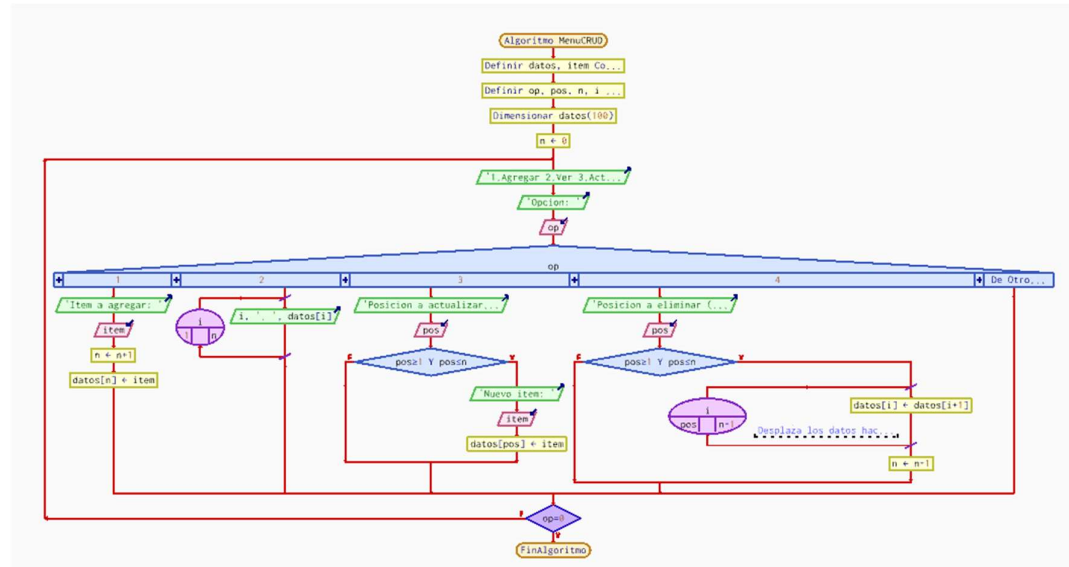
Para i <- pos Hasta n-1 Hacer

datos[i] <- datos[i+1] // Desplaza los datos hacia arriba



FinPara
 $n \leftarrow n - 1$
 FinSi
 FinSegun
 Hasta Que $op = 0$
 FinAlgoritmo

Diagrama de Flujo



Código C++

```
#include <iostream>
#include <vector>
#include <string>
using namespace std;

void mostrar(const vector<string>& lista) {
    if (lista.empty()) { cout << "(Lista vacia)" << endl; return; }
    for (int i = 0; i < lista.size(); i++)
        cout << i << ". " << lista[i] << endl;
}

int main() {
    vector<string> datos;
    int opcion; string item;

    do {
        cout << "\n=== MENU CRUD ===" << endl;
        cout << "1. Agregar " << endl;
        cout << "2. Mostrar " << endl;
        cout << "3. Actualizar" << endl;
        cout << "4. Eliminar" << endl;
        cout << "0. Salir" << endl;
        cout << "Opcion: "; cin >> opcion;
        if (opcion == 1) {
            cout << "Ingrese item: "; cin >> item;
```



```
        datos.push_back(item);
        cout << "Agregado correctamente." << endl;
    } else if (opcion == 2) {
        mostrar(datos);
    } else if (opcion == 3) {
        mostrar(datos);
        int pos; cout << "Posicion a actualizar: "; cin >> pos;
        if (pos >= 0 && pos < datos.size()) {
            cout << "Nuevo valor: "; cin >> item;
            datos[pos] = item;
        } else cout << "Posicion invalida." << endl;
    } else if (opcion == 4) {
        mostrar(datos);
        int pos; cout << "Posicion a eliminar: "; cin >> pos;
        if (pos >= 0 && pos < datos.size())
            datos.erase(datos.begin() + pos);
        else cout << "Posicion invalida." << endl;
    }
} while (opcion != 0);
return 0;
}
```

Prueba de Escritorio

```
=== MENU CRUD ===
1. Agregar
2. Mostrar
3. Actualizar
4. Eliminar
0. Salir
Opcion: 1
Ingrese item: Curacion
Agregado correctamente.

=== MENU CRUD ===
1. Agregar
2. Mostrar
3. Actualizar
4. Eliminar
0. Salir
Opcion: 4
0. Curacion
Posicion a eliminar: 0

=== MENU CRUD ===
1. Agregar
2. Mostrar
3. Actualizar
4. Eliminar
0. Salir
Opcion: 2
(Lista vacía)
```

```
=== MENU CRUD ===
1. Agregar
2. Mostrar
3. Actualizar
4. Eliminar
0. Salir
Opcion: 0
PS C:\Users\ready\OneDrive\Documents\PROGRAMACION_UTA\pract
icas>
```

Imagen 13 y 14 Evidencia C++ hecha en VisualCode

Código Java

```
import java.util.ArrayList;
import java.util.Scanner;

public class Ejercicio4 {
    static ArrayList<String> datos = new ArrayList<>();
    static Scanner sc = new Scanner(System.in);

    static void mostrar() {
        if (datos.isEmpty()) { System.out.println("(Lista vacía)"); return; }
        for (int i = 0; i < datos.size(); i++)
            System.out.println(i + ". " + datos.get(i));
    }
}
```




```
public static void main(String[] args) {
    int opcion;
    do {
        System.out.println("\n=== MENU CRUD ===");
        System.out.println("1.Agregar 2.Mostrar 3.Actualizar 4.Eliminar 0.Salir");
        System.out.print("Opcion: ");
        opcion = sc.nextInt();

        if (opcion == 1) {
            System.out.print("Ingrese item: ");
            datos.add(sc.next());
            System.out.println("Agregado correctamente.");
        } else if (opcion == 2) {
            mostrar();
        } else if (opcion == 3) {
            mostrar();
            System.out.print("Posicion a actualizar: ");
            int pos = sc.nextInt();
            if (pos >= 0 && pos < datos.size()) {
                System.out.print("Nuevo valor: ");
                datos.set(pos, sc.next());
            } else System.out.println("Posicion invalida.");
        } else if (opcion == 4) {
            mostrar();
            System.out.print("Posicion a eliminar: ");
            int pos = sc.nextInt();
            if (pos >= 0 && pos < datos.size()) datos.remove(pos);
            else System.out.println("Posicion invalida.");
        }
    } while (opcion != 0);
    sc.close();
}
```

Prueba de Escritorio

```
ednat.java\jdt_ws\practicas_d5ab0d49\bin\ 'Ejercicio4'

=== MENU CRUD ===
1.Agregar 2.Mostrar 3.Actualizar 4.Eliminar 0.Salir
Opcion: 1
Ingrese item: Lapis
Agregado correctamente.

=== MENU CRUD ===
1.Agregar 2.Mostrar 3.Actualizar 4.Eliminar 0.Salir
Opcion: 3
0. Lapis
Posicion a actualizar: 4
Posicion invalida.

=== MENU CRUD ===
1.Agregar 2.Mostrar 3.Actualizar 4.Eliminar 0.Salir
Opcion: 4
0. Lapis
Posicion a eliminar: 0

=== MENU CRUD ===
1.Agregar 2.Mostrar 3.Actualizar 4.Eliminar 0.Salir
Opcion: 2
(Lista vacia)

=== MENU CRUD ===
1.Agregar 2.Mostrar 3.Actualizar 4.Eliminar 0.Salir
Opcion: 0
PS C:\Users\ready\OneDrive\Documents\PROGRAMACION_UTA\practicas>

C:\Users\ready\OneDrive\Documents\PROGRAMACION_UTA\practicas> java -cp bin\practicas_d5ab0d49\bin\ 'Ejercicio5'

=== MENU CRUD ===
1.Agregar 2.Mostrar 3.Actualizar 4.Eliminar 0.Salir
Opcion: 1
Ingrese item: Manzana
Agregado correctamente.

=== MENU CRUD ===
1.Agregar 2.Mostrar 3.Actualizar 4.Eliminar 0.Salir
Opcion: 1
Ingrese item: Lapis
Agregado correctamente.

=== MENU CRUD ===
1.Agregar 2.Mostrar 3.Actualizar 4.Eliminar 0.Salir
Opcion: 3
0. Manzana
1. Lapis
Posicion a actualizar: 0
Nuevo valor: 1

=== MENU CRUD ===
1.Agregar 2.Mostrar 3.Actualizar 4.Eliminar 0.Salir
Opcion: 0
PS C:\Users\ready\OneDrive\Documents\PROGRAMACION_UTA\practicas>
```

Imagen 15 y 16 Evidencia Java hecha en VisualCode

Pruebas de Escritorio



Caso	Entrada	Proceso	Salida Esperada
1	op=1: Manzana; op=1: Pera; op=2	add x2, recorrer	0.Manzana 1.Pera
2	op=3: pos=0, nuevo=Uva; op=2	set(0,'Uva')	0.Uva 1.Pera
3	op=4: pos=1; op=2	remove(1)	0.Uva

Ejercicio 5 – ArrayList II – Nombres y Notas con Estado

Enunciado del Problema

Registrar en una lista dinámica los nombres y notas de n estudiantes. El programa debe mostrar el listado completo indicando, para cada estudiante, si aprobó (nota ≥ 7) o reprobó.

Análisis del Problema

El ejercicio combina el almacenamiento paralelo de dos tipos de datos (String y double) en listas sincronizadas, o bien mediante un struct/clase. La lógica de evaluación del estado de cada estudiante se aplica durante la presentación de resultados.

Variables de Entrada y Salida

Entrada

- n: cantidad total de alumnos a registrar (Entero).
- nombres[n]: arreglo para almacenar los nombres de los estudiantes (Caracter).
- notas[n]: arreglo paralelo para almacenar las calificaciones (Real).

Proceso

- Leer la variable n.
- Ejecutar un ciclo Para de 1 hasta n para pedir y guardar simultáneamente el nombre y la nota en sus respectivos arreglos (en la misma posición i).
- Recorrer la lista con otro ciclo para evaluar una condición lógica: si notas[i] ≥ 7 , se asigna internamente el estado de aprobación, caso contrario, de reprobación.

Salida

- Lista concatenada impresa en pantalla que muestra: nombre del estudiante, un guion, y el texto "APROBADO" o "REPROBADO" según corresponda.

Variable	Tipo de Dato	Descripción	E / S
Nombre	String / string	Nombre del estudiante	Entrada
Nota	double / float	Calificación del estudiante	Entrada
N	Int	Cantidad de estudiantes	Entrada
Estado	String / string	APROBADO o REPROBADO	Salida

Algoritmo

1. Inicio
2. Solicitar la cantidad n de estudiantes.
3. Ciclo n veces: leer nombre y nota, almacenar en listas paralelas o struct.



4. Recorrer la lista.
5. Para cada estudiante: si nota $\geq 7 \rightarrow$ APROBADO, sino $\rightarrow$ REPROBADO.
6. Mostrar nombre, nota y estado.
7. Fin del programa.

Pseudocódigo (PSeInt)

Algoritmo NombresNotas

Definir nombres Como Caracter

Definir notas Como Real

Definir n, i Como Entero

Escribir Sin Saltar "Cantidad de alumnos: "

Leer n

Dimension nombres[n], notas[n]

Para i <- 1 Hasta n Hacer

 Escribir Sin Saltar "Nombre del alumno ", i, ": "

 Leer nombres[i]

 Escribir Sin Saltar "Nota de ", nombres[i], ": "

 Leer notas[i]

FinPara

Escribir "--- Resultados ---"

Para i <- 1 Hasta n Hacer

 Si notas[i] ≥ 7 Entonces

 Escribir nombres[i], " - APROBADO"

 SiNo

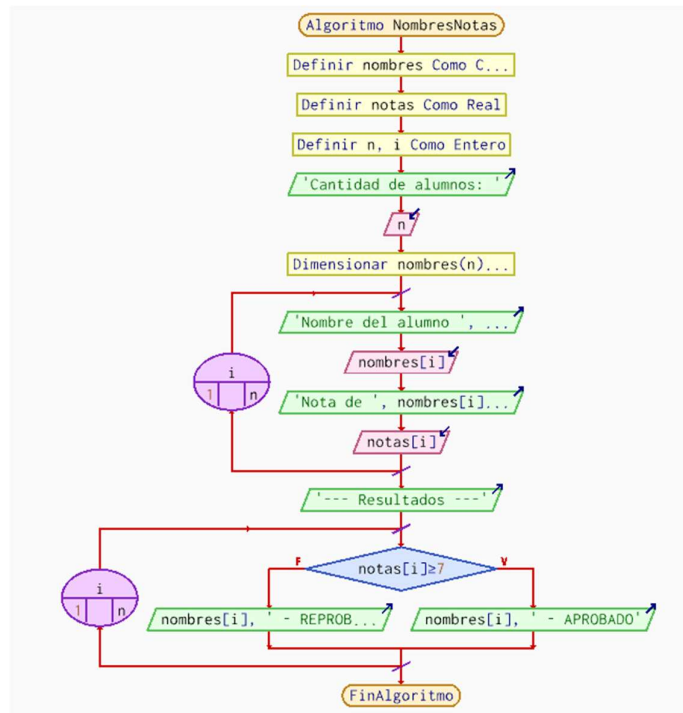
 Escribir nombres[i], " - REPROBADO"

 FinSi

FinPara

FinAlgoritmo

Diagrama de Flujo



Código C++

```
#include <iostream>
#include <vector>
#include <string>
using namespace std;
```

```
struct Estudiante { string nombre; float nota; };
```

```
int main() {
    vector<Estudiante> lista;
    int n;
    cout << " Nombres y Notas con Estado " << endl;
    cout << "¿Cuantos estudiantes? "; cin >> n;

    for (int i = 0; i < n; i++) {
        Estudiante e;
        cout << "Nombre: "; cin >> e.nombre;
        cout << "Nota (0-10): "; cin >> e.nota;
        lista.push_back(e);
    }

    cout << "\n--- Listado ---" << endl;
    for (auto& e : lista)
        cout << e.nombre << " - Nota: " << e.nota
            << (e.nota >= 7 ? "[APROBADO]" : "[REPROBADO]") << endl;
    return 0;
}
```

Prueba de Escritorio



```
g:\msys64\usr\bin\gcc.exe -c main.cpp
g:\msys64\usr\bin\gdb.exe --interpreter=mi'
Nombres y Notas con Estado
¿Cuántos estudiantes? 2
Nombre: Alexander
Nota (0-10): 10
Nombre: Ian
Nota (0-10): 10
--- Listado ---
Alexander - Nota: 10 [APROBADO]
Ian - Nota: 10 [APROBADO]
PS C:\Users\ready\OneDrive\Documents\PROGRAMACION_UTA\practicas>

g:\msys64\usr\bin\gdb.exe --interpreter=mi'
Nombres y Notas con Estado
¿Cuántos estudiantes? 3
Nombre: 3
Nota (0-10): 10
Nombre: Ali
Nota (0-10): 9
Nombre: Solis
Nota (0-10): 6
--- Listado ---
3 - Nota: 10 [APROBADO]
Ali - Nota: 9 [APROBADO]
Solis - Nota: 6 [REPROBADO]
PS C:\Users\ready\OneDrive\Documents\PROGRAMACION_UTA\practicas>
```

Imagen 17 y 18 Evidencia C++ hecha en VisualCode

Código Java

```
import java.util.ArrayList;
import java.util.Scanner;

public class Ejercicio5 {
    public static void main(String[] args) {
        ArrayList<String> nombres = new ArrayList<>();
        ArrayList<Double> notas = new ArrayList<>();
        Scanner sc = new Scanner(System.in);

        System.out.print("¿Cuántos estudiantes? ");
        int n = sc.nextInt();

        for (int i = 0; i < n; i++) {
            System.out.print("Nombre: "); nombres.add(sc.next());
            System.out.print("Nota (0-10): "); notas.add(sc.nextDouble());
        }

        System.out.println("\n--- Listado ---");
        for (int i = 0; i < nombres.size(); i++) {
            String estado = notas.get(i) >= 7 ? "APROBADO" : "REPROBADO";
            System.out.println(nombres.get(i) + " - " + notas.get(i) + " [" + estado + "]");
        }
        sc.close();
    }
}
```

Prueba de Escritorio

```
83541256\redhat.java\jdk_ws\practicas_d5ab0d49\bin\ 'Ejercicio5'
¿Cuántos estudiantes? 3
Nombre: Ian
Nota (0-10): 9
Nombre: Jhair
Nota (0-10): 10
Nombre: Alex
Nota (0-10): 10
--- Listado ---
Ian - 9.0 [APROBADO]
Jhair - 10.0 [APROBADO]
Alex - 10.0 [APROBADO]
PS C:\Users\ready\OneDrive\Documents\PROGRAMACION_UTA\practicas>

83541256\redhat.java\jdk_ws\practicas_d5ab0d49\bin\ 'Ejercicio5'
¿Cuántos estudiantes? 2
Nombre: Alexander
Nota (0-10): 9
Nombre: Selene
Nota (0-10): 6
--- Listado ---
Alexander - 9.0 [APROBADO]
Selene - 6.0 [REPROBADO]
PS C:\Users\ready\OneDrive\Documents\PROGRAMACION_UTA\practicas>
```

Imagen 19 y 20 Evidencia Java hecha en VisualCode

Pruebas de Escritorio



Caso	Entrada	Proceso	Salida Esperada
1	Ana/9.0, Luis/5.0, Eva/7.0	$9 \geq 7 \checkmark, 5 < 7 \times, 7 \geq 7 \checkmark$	Ana APROBADO, Luis REPROBADO, Eva APROBADO
2	Carlos/3.5, María/8.2	$3.5 < 7 \times, 8.2 \geq 7 \checkmark$	Carlos REPROBADO, María APROBADO
3	Pedro/10, Juan/6.9	$10 \geq 7 \checkmark, 6.9 < 7 \times$	Pedro APROBADO, Juan REPROBADO

Ejercicio 6 – ArrayList II – Sistema de Inventario

Enunciado del Problema

Crear un sistema de inventario usando ArrayList/vector que permita agregar productos con nombre, cantidad y precio, visualizar el inventario completo y eliminar productos por posición mediante un menú interactivo.

Análisis del Problema

El sistema gestiona tres atributos por producto. En Java se usan tres ArrayList paralelos; en C++ se usa un vector de structs. Las operaciones de eliminación deben sincronizar las tres listas/atributos simultáneamente.

Variables de Entrada y Salida

Entrada

- op: número elegido en el menú de opciones (Entero).
- Tres arreglos paralelos (nombres[100], cantidades[100], precios[100]) para almacenar los atributos del inventario.
- Variables temporales para leer los datos del usuario: nombre (Caracter), cantidad (Entero), precio (Real), y pos (posición a eliminar, Entero).

Proceso

- Control mediante un ciclo Repetir basado en la opción op.
- **Agregar (1):** Incrementar el contador de productos n en 1 y guardar los tres atributos en el índice n de los tres arreglos paralelos.
- **Eliminar (3):** Leer la posición pos, validar que exista, y usar un ciclo para desplazar los datos de los *tres arreglos* un índice hacia atrás, sobrescribiendo el registro eliminado y reduciendo n en 1.

Salida

- **Ver (2):** Reporte completo en pantalla mostrando el índice, nombre, cantidad y precio formateado (con signo \$) de todos los productos vigentes.

Variable	Tipo de Dato	Descripción	E / S
Nombre	String / string	Nombre del producto	Entrada
Cantidad	Int	Unidades disponibles en inventario	Entrada
Precio	double / float	Precio unitario del producto	Entrada
Pos	Int	Posición del producto a eliminar	Entrada
Inventario mostrado	Texto formateado	Lista completa del inventario	Salida

Algoritmo



1. Inicio
2. Declarar listas para nombre, cantidad y precio.
3. Ciclo do-while con menú: 1-Agregar, 2-Ver, 3-Eliminar, 0-Salir.
4. Agregar: leer los tres atributos y agregarlos a sus respectivas listas.
5. Ver: recorrer las listas e imprimir cada producto con sus datos.
6. Eliminar: mostrar inventario, pedir posición, validar y eliminar de las tres listas.
7. Repetir hasta que el usuario elija salir.
8. Fin

Pseudocódigo (PSeInt)

Algoritmo Inventario

Definir nombres Como Caracter

Definir cantidades, op, pos, n, i Como Entero

Definir precios Como Real

Dimension nombres[100], cantidades[100], precios[100]

n <- 0

Repetir

Escribir "1.Agregar 2.Ver 3.Eliminar 0.Salir"

Escribir Sin Saltar "Opcion: "

Leer op

Si op = 1 Entonces

n <- n + 1

Escribir Sin Saltar "Nombre: "; Leer nombres[n]

Escribir Sin Saltar "Cantidad: "; Leer cantidades[n]

Escribir Sin Saltar "Precio: "; Leer precios[n]

FinSi

Si op = 2 Entonces

Para i <- 1 Hasta n Hacer

Escribir i, ". ", nombres[i], " | Cant: ", cantidades[i], " | Prec: \$", precios[i]

FinPara

FinSi

Si op = 3 Entonces

Escribir Sin Saltar "Posicion a eliminar: "; Leer pos

Si pos >= 1 Y pos <= n Entonces

Para i <- pos Hasta n-1 Hacer

nombres[i] <- nombres[i+1]

cantidades[i] <- cantidades[i+1]

precios[i] <- precios[i+1]

FinPara

n <- n - 1

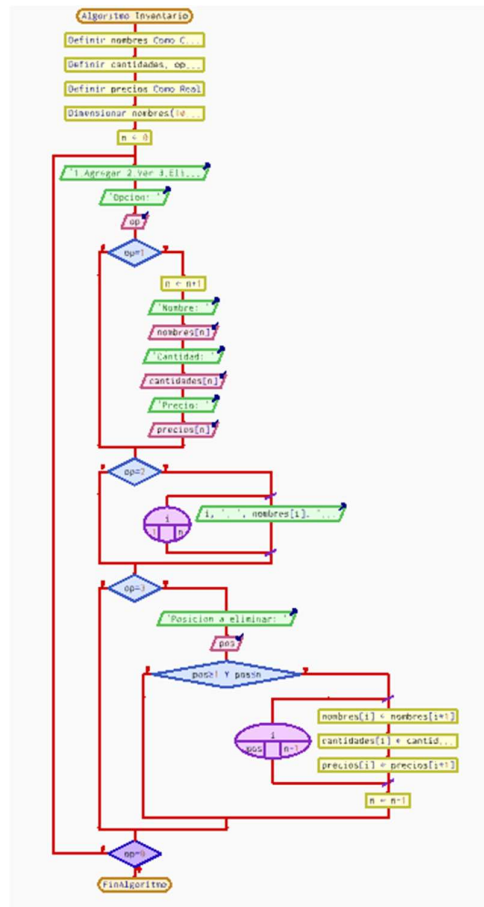
FinSi

FinSi

Hasta Que op = 0

FinAlgoritmo

Diagrama de Flujo



Código C++

```
#include <iostream>
#include <vector>
#include <string>
using namespace std;
```

```
struct Producto { string nombre; int cantidad; float precio; };
```

```
void mostrarInventario(const vector<Producto>& inv) {
    cout << "\n--- INVENTARIO ---" << endl;
    for (auto& p : inv)
        cout << p.nombre << " | Cant: " << p.cantidad
            << " | Precio: $" << p.precio << endl;
}
```

```
int main() {
    vector<Producto> inventario;
    int op;
    do {
        cout << "ArrayList II – Sistema de Inventario" << endl;
        cout << "1.Agregar" << endl;
        cout << "2.Ver" << endl;
        cout << "3.Eliminar" << endl;
        cout << "0.Salir -> " << endl;
        cin >> op;
```




```
if (op == 1) {
    Producto pr;
    cout << "Nombre: "; cin >> pr.nombre;
    cout << "Cantidad: "; cin >> pr.cantidad;
    cout << "Precio: "; cin >> pr.precio;
    inventario.push_back(pr);
} else if (op == 2) {
    mostrarInventario(inventario);
} else if (op == 3) {
    mostrarInventario(inventario);
    int pos; cout << "Posicion a eliminar: "; cin >> pos;
    if (pos >= 0 && pos < inventario.size())
        inventario.erase(inventario.begin() + pos);
    }
} while (op != 0);
return 0;
}
```

Prueba de Escritorio

```
ArrayList II - Sistema de Inventario
1.Agregar
2.Ver
3.Eliminar
0.Salir ->
1
Nombre: Queso
Cantidad: 2
Precio: 1
ArrayList II - Sistema de Inventario
1.Agregar
2.Ver
3.Eliminar
0.Salir ->
2
--- INVENTARIO ---
Queso | Cant: 2 | Precio: $1
ArrayList II - Sistema de Inventario
1.Agregar
2.Ver
3.Eliminar
0.Salir ->
2
--- INVENTARIO ---
Queso | Cant: 2 | Precio: $1
ArrayList II - Sistema de Inventario
1.Agregar
2.Ver
3.Eliminar
0.Salir ->
0

Interpreter=ml
ArrayList II - Sistema de Inventario
1.Agregar
2.Ver
3.Eliminar
0.Salir ->
1
Nombre: Pan
Cantidad: 10
Precio: 0.5
ArrayList II - Sistema de Inventario
1.Agregar
2.Ver
3.Eliminar
0.Salir ->
3
--- INVENTARIO ---
Pan | Cant: 10 | Precio: $0.5
Posicion a eliminar: 0
ArrayList II - Sistema de Inventario
1.Agregar
2.Ver
3.Eliminar
0.Salir ->
0
PS C:\Users\ready\OneDrive\Documents\PROGRAMACION_UTA\pract
icas>
```

Imagen 23 y 24 Evidencia C++ hecha en VisualCode

Código Java

```
import java.util.ArrayList;
import java.util.Scanner;

public class Ejercicio6 {
    static ArrayList<String> nombres = new ArrayList<>();
    static ArrayList<Integer> cantidades = new ArrayList<>();
    static ArrayList<Double> precios = new ArrayList<>();
    static Scanner sc = new Scanner(System.in);

    static void mostrar() {
        System.out.println("\n--- INVENTARIO ---");
        for (int i = 0; i < nombres.size(); i++)
```



```
System.out.println(i+" "+nombres.get(i)+" | Cant: "+cantidades.get(i)+" |  
$"+precios.get(i));  
}  
  
public static void main(String[] args) {  
    int op;  
    do {  
        System.out.print("1.Agregar 2.Ver 3.Eliminar 0.Salir -> ");  
        op = sc.nextInt();  
        if (op == 1) {  
            System.out.print("Nombre: "); nombres.add(sc.next());  
            System.out.print("Cantidad: "); cantidades.add(sc.nextInt());  
            System.out.print("Precio: "); precios.add(sc.nextDouble());  
        } else if (op == 2) {  
            mostrar();  
        } else if (op == 3) {  
            mostrar();  
            System.out.print("Posicion: "); int pos = sc.nextInt();  
            if (pos >= 0 && pos < nombres.size()) {  
                nombres.remove(pos); cantidades.remove(pos); precios.remove(pos);  
            }  
        }  
    } while (op != 0);  
    sc.close();  
}
```

Prueba de Escritorio

```
780eacc83541256\redhat.java\jdt_ws\practicass_d5ab0d49\bin' 'Ejercicio6'  
1.Agregar 2.Ver 3.Eliminar 0.Salir -> 1  
Nombre: Leche  
Cantidad: 2  
Precio: 1.50  
1.Agregar 2.Ver 3.Eliminar 0.Salir -> 3  
  
--- INVENTARIO ---  
0. Leche | Cant: 2 | $1.5  
Posicion: 0  
1.Agregar 2.Ver 3.Eliminar 0.Salir -> 2  
  
--- INVENTARIO ---  
1.Agregar 2.Ver 3.Eliminar 0.Salir -> 0  
PS C:\Users\ready\OneDrive\Documents\PROGRAMACION_UTA\pract  
icas>  
  
33541256\redhat.java\jdt_ws\practicass_d5ab0d49\bin' 'Ejercicio6'  
1.Agregar 2.Ver 3.Eliminar 0.Salir -> 1  
Nombre: Queso  
Cantidad: 5  
Precio: 3  
1.Agregar 2.Ver 3.Eliminar 0.Salir -> 3  
  
--- INVENTARIO ---  
0. Queso | Cant: 5 | $3.0  
Posicion: 1  
1.Agregar 2.Ver 3.Eliminar 0.Salir -> 2  
  
--- INVENTARIO ---  
0. Queso | Cant: 5 | $3.0  
1.Agregar 2.Ver 3.Eliminar 0.Salir -> 0  
PS C:\Users\ready\OneDrive\Documents\PROGRAMACION_UTA\practicass>
```

Imagen 23 y 24 Evidencia Java hecha en VisualCode

Pruebas de Escritorio

Caso	Entrada	Proceso	Salida Esperada
1	op=1: Leche/10/\$1.50; op=2	push_back x3 atribs; mostrar	Leche Cant:10 \$1.50
2	op=1: Pan/20/\$0.80; op=1: Queso/5/\$3.00; op=3: pos=1	erase pos=1	Quedan: Leche, Queso



3	op=2 en lista vacía	isEmpty() = true	(Lista vacía)
---	---------------------	------------------	---------------

Ejercicio 7 – Arreglo vs ArrayList – Comparación Estructural

Enunciado del Problema

Comparar el uso de arreglos estáticos frente a ArrayList/vector en Java y C++. El programa debe crear ambas estructuras con los mismos 10 elementos enteros del 1 al 10, mostrar su contenido y presentar una comparación textual de sus características principales.

Análisis del Problema

Los arreglos estáticos tienen tamaño fijo definido en tiempo de compilación, mientras que ArrayList/vector son dinámicos y ofrecen métodos de alto nivel. Ambas estructuras permiten acceso por índice, pero difieren en flexibilidad y funcionalidades.

Variables de Entrada y Salida

Entrada

- No requiere datos ingresados por el usuario por teclado. Las variables arreglo[10] y lista[10] (que simula el array dinámico) son definidas internamente por el algoritmo.

Proceso

- Utilizar un ciclo iterativo Para que vaya de 1 hasta 10.
- Asignar secuencialmente a cada posición i de ambas estructuras el valor numérico del mismo índice (i).

Salida

- Impresión visual del contenido de ambas estructuras, comprobando que almacenan la misma serie numérica (1 al 10).
- Mensajes de texto estáticos que explican teóricamente las diferencias conceptuales (Arreglo: tamaño fijo, acceso directo vs Lista: tamaño dinámico, métodos).

Variable	Tipo de Dato	Descripción	E / S
arreglo[]	int[] / int[10]	Arreglo estático con 10 elementos	Proceso
lista/vec	ArrayList<Integer> / vector<int>	Lista dinámica con 10 elementos	Proceso
Comparación	Texto	Diferencias impresas en pantalla	Salida

Algoritmo

1. Inicio
2. Declarar un arreglo estático de tamaño 10 con valores 1-10.
3. Declarar un ArrayList/vector vacío.
4. Ciclo: agregar los valores 1-10 al ArrayList/vector.
5. Mostrar los elementos del arreglo.
6. Mostrar los elementos del ArrayList/vector.
7. Imprimir la comparación de características.
8. Fin

Pseudocódigo (PSeInt)



```
Algoritmo ArregloVsArrayList
// 1. Definición de variables y estructuras
Definir i Como Entero

// En PSeInt las dimensiones se declaran por separado
Dimension arreglo[10]
Dimension lista[10] // Simulamos la lista con otro arreglo para fines didácticos

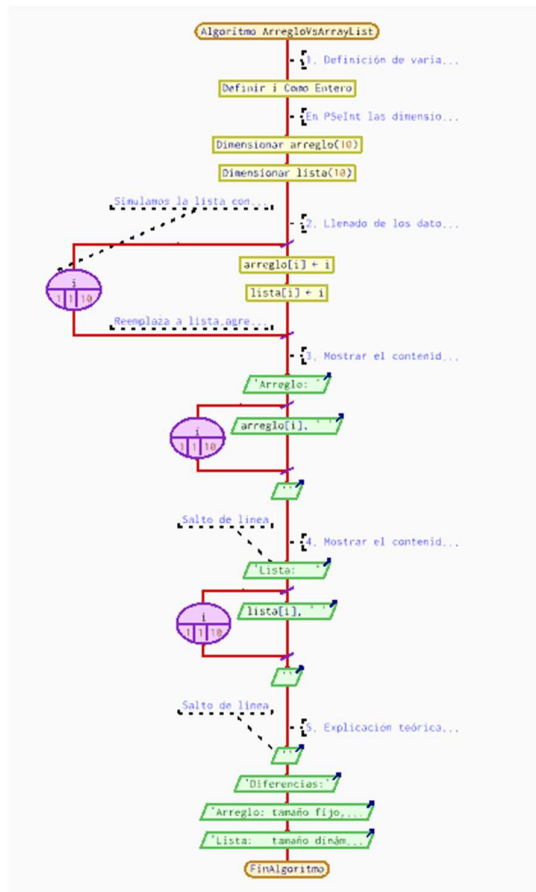
// 2. Llenado de los datos (Índices en PSeInt empiezan en 1)
Para i <- 1 Hasta 10 Con Paso 1 Hacer
    arreglo[i] <- i
    lista[i] <- i // Reemplaza a lista.agregar()
FinPara

// 3. Mostrar el contenido del Arreglo
Escribir Sin Saltar "Arreglo: "
Para i <- 1 Hasta 10 Con Paso 1 Hacer
    Escribir Sin Saltar arreglo[i], " "
FinPara
Escribir "" // Salto de línea

// 4. Mostrar el contenido de la Lista
Escribir Sin Saltar "Lista: "
Para i <- 1 Hasta 10 Con Paso 1 Hacer
    Escribir Sin Saltar lista[i], " "
FinPara
Escribir "" // Salto de línea

// 5. Explicación teórica de las diferencias
Escribir ""
Escribir "Diferencias:"
Escribir "Arreglo: tamaño fijo, acceso directo"
Escribir "Lista: tamaño dinámico, métodos add/remove"
FinAlgoritmo
```

Diagrama de Flujo



Código C++

```
#include <iostream>
#include <vector>
#include <chrono>
using namespace std;
using namespace std::chrono;

int main() {
    const int N = 1000000;

    // --- Arreglo estático ---
    auto t1 = high_resolution_clock::now();
    int arreglo[10] = {1,2,3,4,5,6,7,8,9,10};
    cout << "Arreglo (primeros 10 elementos):" << endl;
    for (int i = 0; i < 10; i++) cout << arreglo[i] << " ";
    cout << endl;
    auto t2 = high_resolution_clock::now();

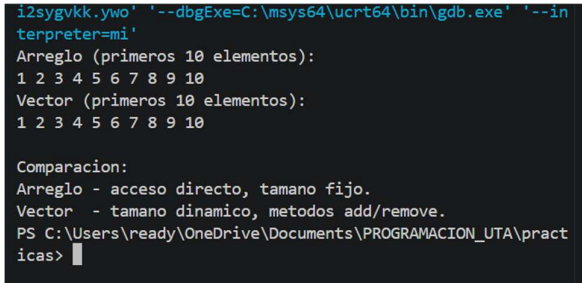
    // --- Vector (ArrayList equivalente) ---
    vector<int> vec;
    for (int i = 1; i <= 10; i++) vec.push_back(i);
    cout << "Vector (primeros 10 elementos):" << endl;
    for (int v : vec) cout << v << " ";
    cout << endl;
```



```
auto t3 = high_resolution_clock::now();

cout << "\nComparacion:" << endl;
cout << "Arreglo - acceso directo, tamaño fijo." << endl;
cout << "Vector - tamaño dinámico, métodos add/remove." << endl;
return 0;
}
```

Prueba de Escritorio



```
i2sygvkk.ywo' '--dbgExe=C:\msys64\ucrt64\bin\gdb.exe' '--in
terpreter=mi'
Arreglo (primeros 10 elementos):
1 2 3 4 5 6 7 8 9 10
Vector (primeros 10 elementos):
1 2 3 4 5 6 7 8 9 10

Comparacion:
Arreglo - acceso directo, tamaño fijo.
Vector - tamaño dinámico, métodos add/remove.
PS C:\Users\ready\OneDrive\Documents\PROGRAMACION_UTA\pract
icas>
```

Imagen 25 Evidencia C++ hecha en VisualCode

Código Java

```
import java.util.ArrayList;

public class Ejercicio7 {
    public static void main(String[] args) {
        // --- Arreglo estático ---
        int[] arreglo = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
        System.out.print("Arreglo: ");
        for (int v : arreglo) System.out.print(v + " ");
        System.out.println();

        // --- ArrayList dinámico ---
        ArrayList<Integer> lista = new ArrayList<>();
        for (int i = 1; i <= 10; i++) lista.add(i);
        System.out.print("ArrayList: ");
        for (int v : lista) System.out.print(v + " ");
        System.out.println();

        System.out.println("\nComparacion:");
        System.out.println("Arreglo : tamaño fijo, acceso por índice [i].");
        System.out.println("ArrayList : tamaño dinámico, métodos add/remove/get.");
    }
}
```

Prueba de Escritorio



```
780eacc83541256\redhat.java\jdt_ws\practicass_d5ab0d49\bin'  
'Ejercicio7'  
Arreglo: 1 2 3 4 5 6 7 8 9 10  
ArrayList: 1 2 3 4 5 6 7 8 9 10  
  
Comparacion:  
Arreglo : tamaño fijo, acceso por índice [i].  
ArrayList : tamaño dinámico, métodos add/remove/get.  
PS C:\Users\ready\OneDrive\Documents\PROGRAMACION_UTA\pract  
icas>
```

Imagen 26 Evidencia Java hecha en VisualCode

Pruebas de Escritorio

Caso	Entrada	Proceso	Salida Esperada
1	Arreglo = {1..10}	Acceso directo [i]	Arreglo: 1 2 3 4 5 6 7 8 9 10
2	Lista = {1..10}	Métodos add/push_back	ArrayList: 1 2 3 4 5 6 7 8 9 10
3	Comparación	Análisis de características	Arreglo: fijo ArrayList: dinámico

Ejercicio 8 – Arreglo vs ArrayList – Comparación de Inserción

Enunciado del Problema

Comparar el tiempo de inserción de 100.000 elementos en un arreglo estático frente a un ArrayList/vector. El programa debe medir el tiempo en microsegundos para cada estructura y mostrar los resultados comparativos.

Análisis del Problema

La medición de rendimiento permite evidenciar las diferencias de eficiencia entre estructuras. Los arreglos tienen inserción $O(1)$ directa, mientras que ArrayList/vector puede requerir reubicaciones de memoria al crecer. Se usan las APIs de tiempo de alta resolución de cada lenguaje.

Variables de Entrada y Salida

Entrada

- N: Constante numérica predefinida que determina la cantidad de iteraciones (ej. 5000 o 100000).

Proceso

- Declarar el tamaño N.
- Registrar un punto de tiempo inicial t1.
- Ejecutar un ciclo Para N veces para llenar el arreglo (simulado con valores aleatorios o iterativos).
- Registrar el tiempo t2 y calcular la diferencia de la primera prueba.
- Repetir el mismo proceso midiendo los tiempos t3 y t4 para llenar la segunda estructura simulada (lista).

Salida

- Mensajes que indican cuándo inicia y finaliza cada fase de llenado.
- Impresión final de la duración o mensaje de éxito demostrando la finalización del proceso de inserción masiva.



Variable	Tipo de Dato	Descripción	E / S
N	int (constante)	Número de elementos a insertar (100.000)	Proceso
arreglo[N]	int[] / int[N]	Arreglo estático llenado con ciclo	Proceso
lista	ArrayList/vector<int>	Lista dinámica llenada con add/push_back	Proceso
durArr / durVec	long / microseconds	Tiempo de cada operación en microsegundos	Salida

Algoritmo

1. Inicio
2. Definir constante N = 100.000.
3. Registrar tiempo inicial t1.
4. Llenar arreglo con ciclo de N iteraciones.
5. Registrar tiempo final t2; calcular duración del arreglo.
6. Registrar tiempo inicial t3.
7. Llenar ArrayList/vector con add/push_back en ciclo de N iteraciones.
8. Registrar tiempo final t4; calcular duración de la lista.
9. Mostrar ambos tiempos comparativamente.
10. Fin

Pseudocódigo (PSeInt)

Algoritmo ComparacionInsercion

// 1. Definición de variables con sintaxis estricta
Definir N, i Como Entero

N <- 5000 // Tamaño ajustado para evitar sobrecarga en PSeInt

// Dimensionar las estructuras obligatoriamente
Dimension arreglo[N]
Dimension lista[N]

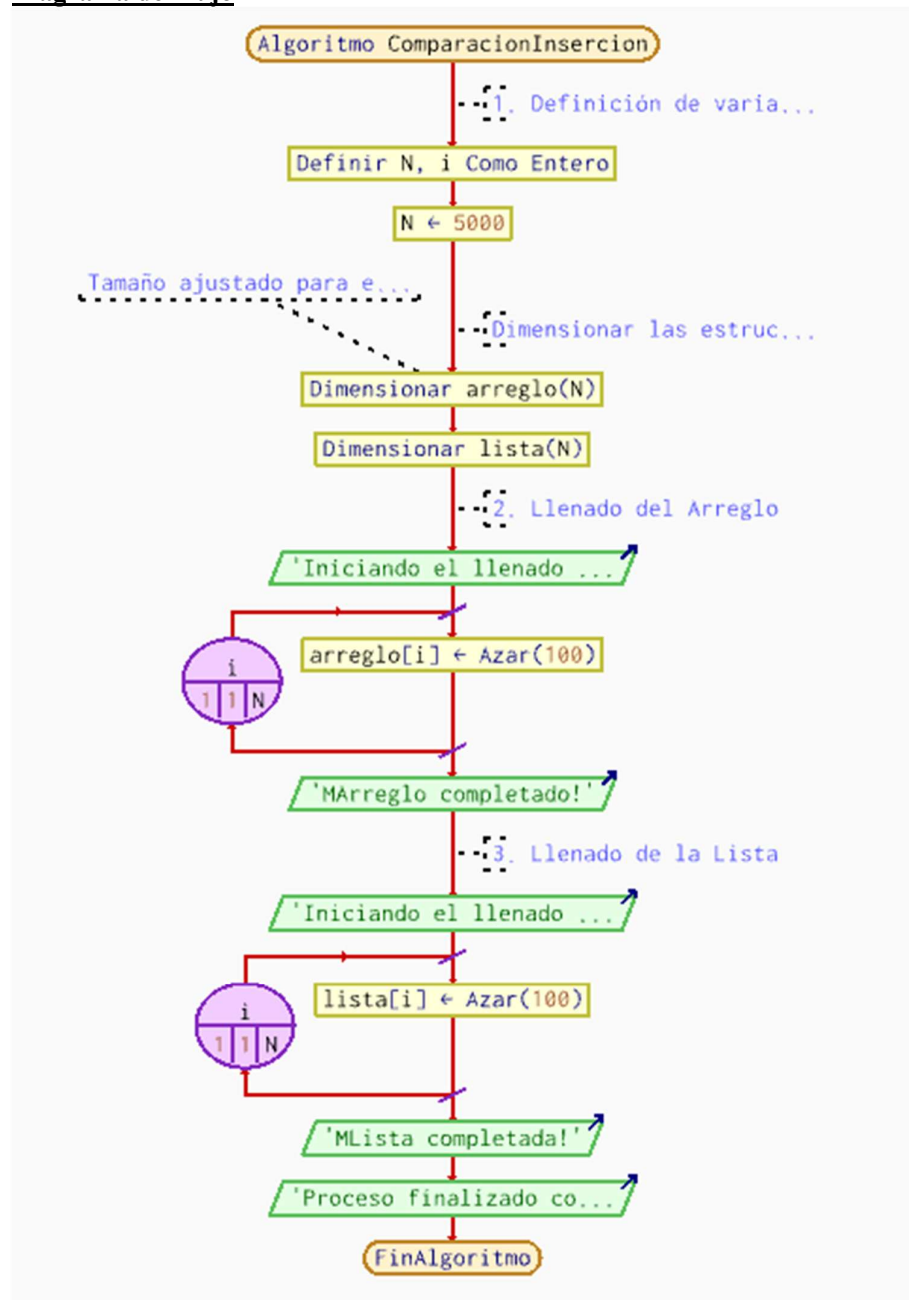
// 2. Llenado del Arreglo
Escribir "Iniciando el llenado del arreglo..."
Para i <- 1 Hasta N Con Paso 1 Hacer
 arreglo[i] <- Azar(100)
FinPara
Escribir "¡Arreglo completado!"

// 3. Llenado de la Lista
Escribir "Iniciando el llenado de la lista..."
Para i <- 1 Hasta N Con Paso 1 Hacer
 lista[i] <- Azar(100)
FinPara
Escribir "¡Lista completada!"

Escribir "Proceso finalizado con éxito."
FinAlgoritmo



Diagrama de Flujo



Código C++

```
#include <iostream>
#include <vector>
#include <chrono>
using namespace std;
using namespace std::chrono;

int main() {
    const int N = 100000;

    // Inserción en arreglo (simulada con índice)
```

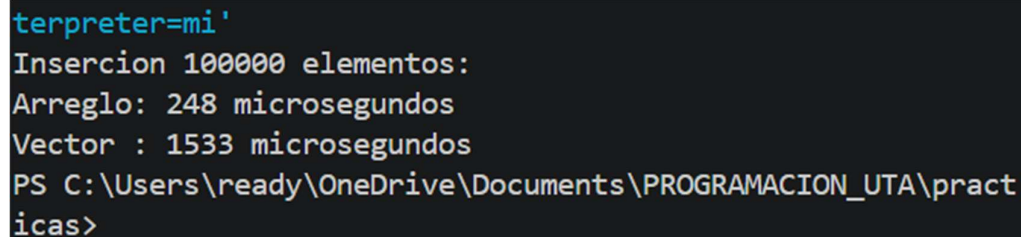


```
int arreglo[N];
auto t1 = high_resolution_clock::now();
for (int i = 0; i < N; i++) arreglo[i] = i;
auto t2 = high_resolution_clock::now();
auto durArr = duration_cast<microseconds>(t2 - t1).count();

// Inserción en vector
vector<int> vec;
auto t3 = high_resolution_clock::now();
for (int i = 0; i < N; i++) vec.push_back(i);
auto t4 = high_resolution_clock::now();
auto durVec = duration_cast<microseconds>(t4 - t3).count();

cout << "Insercion " << N << " elementos:" << endl;
cout << "Arreglo: " << durArr << " microsegundos" << endl;
cout << "Vector : " << durVec << " microsegundos" << endl;
return 0;
}
```

Prueba de Escritorio



```
terpreter=mi'
Insercion 100000 elementos:
Arreglo: 248 microsegundos
Vector : 1533 microsegundos
PS C:\Users\ready\OneDrive\Documents\PROGRAMACION_UTA\practicas>
```

Imagen 27 Evidencia C++ hecha en VisualCode

Código Java

```
import java.util.ArrayList;

public class Ejercicio8 {
    public static void main(String[] args) {
        final int N = 100000;

        // Inserción en arreglo
        int[] arreglo = new int[N];
        long t1 = System.nanoTime();
        for (int i = 0; i < N; i++) arreglo[i] = i;
        long t2 = System.nanoTime();

        // Inserción en ArrayList
        ArrayList<Integer> lista = new ArrayList<>();
        long t3 = System.nanoTime();
        for (int i = 0; i < N; i++) lista.add(i);
        long t4 = System.nanoTime();

        System.out.println("Insercion de " + N + " elementos:");
        System.out.println("Arreglo : " + (t2-t1)/1000 + " microsegundos");
    }
}
```



```
System.out.println("ArrayList: " + (t4-t3)/1000 + " microsegundos");  
}  
}
```

Prueba de Escritorio

```
ppdata (reading code (352) (not repeated) age (5258003244411  
780eacc83541256\redhat.java\jdt_ws\practicad5ab0d49\bin'  
'Ejercicio8'  
Insercion de 100000 elementos:  
Arreglo : 921 microsegundos  
ArrayList: 3475 microsegundos  
PS C:\Users\ready\OneDrive\Documents\PROGRAMACION_UTA\pract  
icas>
```

Imagen 28 Evidencia Java hecha en VisualCode

Pruebas de Escritorio

Caso	Entrada	Proceso	Salida Esperada
1	N=100.000, arreglo	Tiempo medido con chrono/nanoTime	Arreglo: ~200-500 µs
2	N=100.000, ArrayList	push_back/add con reubicaciones	Lista: ~300-800 µs
3	Comparación final	Análisis de resultados	Arreglo generalmente más rápido

Ejercicio 9 – LinkedList – Sistema de Turnos

Enunciado del Problema

Implementar un sistema de gestión de turnos usando LinkedList en Java y list en C++. El programa debe permitir agregar pacientes al final de la fila, atender al primero (eliminarlo, estructura FIFO), y visualizar la lista de turnos pendientes mediante un menú interactivo.

Análisis del Problema

La LinkedList es la estructura ideal para una cola FIFO (First In, First Out). Los métodos addLast/push_back agregan al final y removeFirst/pop_front eliminan del principio, simulando fielmente la dinámica de una sala de espera. Se requiere validar que la lista no esté vacía antes de atender.

Variables de Entrada y Salida

Entrada

- op: opción seleccionada en el menú (Entero).
- nombre: nombre del paciente (Caracter).
- turnos[100]: arreglo utilizado para simular el comportamiento de una cola FIFO.

Proceso

- Control general a través de un ciclo Repetir.
- Mantener dos contadores: n (límite final de la fila) y frente (puntero al inicio de la fila, inicializado en 1).
- **Agregar (1):** Incrementar n y añadir el nombre al final del arreglo.



- **Atender (2):** Validar si hay turnos pendientes evaluando que frente $\leq n$. De ser así, se simula la eliminación (atención) sumando +1 al puntero frente, lo que "avanza" la fila.

Salida

- Impresión de "Atendiendo a: [nombre]" extrayendo el dato apuntado por frente.
- **Ver (3):** Lista secuencial de los pacientes en espera calculada desde la posición actual del frente hasta n.

Variable	Tipo de Dato	Descripción	E / S
Nombre	String / string	Nombre del paciente	Entrada
Opcion	Int	Opción del menú	Entrada
Turnos	LinkedList<String> / list<string>	Cola de turnos pendientes	Proceso
Turno atendido	String	Nombre del paciente atendido	Salida

Algoritmo

1. Inicio
2. Declarar LinkedList/list vacía para turnos.
3. Ciclo do-while con menú: 1-Agregar turno, 2-Atender, 3-Ver turnos, 0-Salir.
4. Agregar: leer nombre y hacer addLast/push_back.
5. Atender: verificar que no esté vacía, mostrar el primero y hacer removeFirst/pop_front.
6. Ver: recorrer e imprimir todos los turnos pendientes.
7. Repetir hasta que el usuario elija salir.
8. Fin

Pseudocódigo (PSeInt)

Algoritmo SistemaTurnos

Definir turnos, nombre Como Caracter

Definir op, n, frente, i Como Entero

Dimension turnos[100]

n <- 0

frente <- 1 // Puntero al primer turno

Repetir

Escribir "1.Agregar 2.Atender 3.Ver 0.Salir"

Escribir Sin Saltar "Opcion: "; Leer op

Si op=1 Entonces

n <- n + 1

Escribir Sin Saltar "Nombre: "; Leer nombre
turnos[n] <- nombre

FinSi

Si op=2 Entonces

Si frente $\leq n$ Entonces

Escribir "Atendiendo a: ", turnos[frente]

frente <- frente + 1 // Avanza la fila

SiNo

Escribir "No hay turnos."

FinSi

FinSi

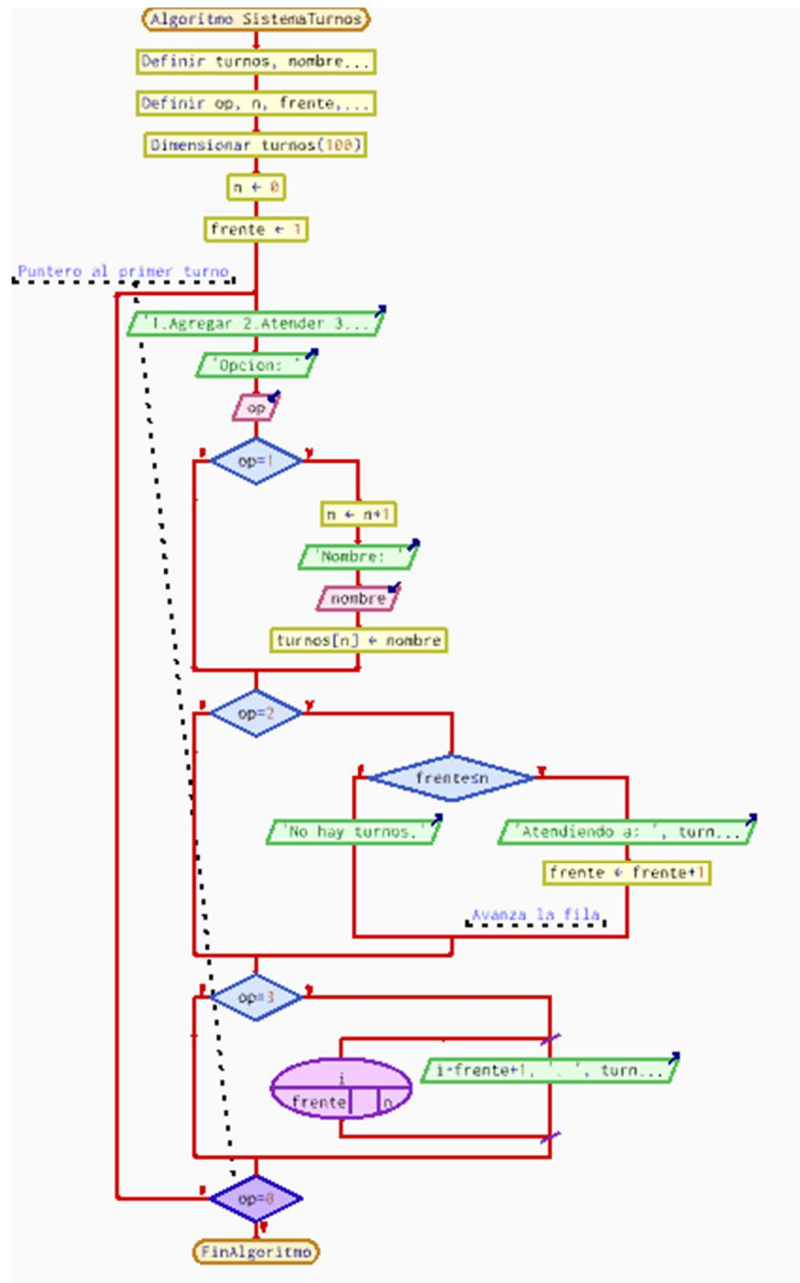
Si op=3 Entonces

Para i <- frente Hasta n Hacer



Escribir i - frente + 1, ". ", turnos[i]
FinPara
FinSi
Hasta Que op = 0
FinAlgoritmo

Diagrama de Flujo



Código C++

```
#include <iostream>
#include <list>
#include <string>
using namespace std;
```



```
int main() {
    list<string> turnos;
    int op; string nombre;

    do {
        cout << "\n=== SISTEMA DE TURNOS (LinkedList) ===" << endl;
        cout << "1.Agregar turno 2.Atender (quitar) 3.Ver turnos 0.Salir -> ";
        cin >> op;

        if (op == 1) {
            cout << "Nombre del paciente: "; cin >> nombre;
            turnos.push_back(nombre); // Agrega al final
            cout << "Turno asignado a " << nombre << endl;
        } else if (op == 2) {
            if (!turnos.empty()) {
                cout << "Atendiendo a: " << turnos.front() << endl;
                turnos.pop_front(); // Elimina el primero (FIFO)
            } else cout << "No hay turnos pendientes." << endl;
        } else if (op == 3) {
            cout << "Turnos en espera:" << endl;
            for (auto& t : turnos) cout << " - " << t << endl;
        }
    } while (op != 0);
    return 0;
}
```

Prueba de Escritorio

```
=== SISTEMA DE TURNOS (LinkedList) ===
1.Agregar turno 2.Atender (quitar) 3.Ver turnos 0.Salir ->1
Nombre del paciente: Jessenia
Turno asignado a Jessenia

=== SISTEMA DE TURNOS (LinkedList) ===
1.Agregar turno 2.Atender (quitar) 3.Ver turnos 0.Salir ->3
Turnos en espera:
- Jessenia

=== SISTEMA DE TURNOS (LinkedList) ===
1.Agregar turno 2.Atender (quitar) 3.Ver turnos 0.Salir ->2
Atendiendo a: Jessenia

=== SISTEMA DE TURNOS (LinkedList) ===
1.Agregar turno 2.Atender (quitar) 3.Ver turnos 0.Salir ->0
PS C:\Users\ready\OneDrive\Documents\PROGRAMACION_UTA\practicas
>

C:\msys64\ucrt64\bin\gdb.exe' '--interpreter=mi'

=== SISTEMA DE TURNOS (LinkedList) ===
1.Agregar turno 2.Atender (quitar) 3.Ver turnos 0.Salir -> 1
Nombre del paciente: Solis
Turno asignado a Solis

=== SISTEMA DE TURNOS (LinkedList) ===
1.Agregar turno 2.Atender (quitar) 3.Ver turnos 0.Salir -> 3
Turnos en espera:
- Solis

=== SISTEMA DE TURNOS (LinkedList) ===
1.Agregar turno 2.Atender (quitar) 3.Ver turnos 0.Salir -> 0
PS C:\Users\ready\OneDrive\Documents\PROGRAMACION_UTA\practicas
>
```

Imagen 29 y 30 Evidencia C++ hecha en VisualCode

Código Java

```
import java.util.LinkedList;
import java.util.Scanner;

public class Ejercicio9 {
    public static void main(String[] args) {
        LinkedList<String> turnos = new LinkedList<>();
        Scanner sc = new Scanner(System.in);
        int op;
```



```
do {
    System.out.println("\n=== SISTEMA DE TURNOS (LinkedList) ===");
    System.out.println("1.Agregar 2.Atender 3.Ver 0.Salir");
    System.out.print("-> ");
    op = sc.nextInt();

    if (op == 1) {
        System.out.print("Nombre del paciente: ");
        String nombre = sc.next();
        turnos.addLast(nombre); // Agregar al final
        System.out.println("Turno asignado a " + nombre);
    } else if (op == 2) {
        if (!turnos.isEmpty()) {
            System.out.println("Atendiendo a: " + turnos.removeFirst());
        } else System.out.println("No hay turnos pendientes.");
    } else if (op == 3) {
        System.out.println("Turnos en espera:");
        for (String t : turnos) System.out.println(" - " + t);
    }
} while (op != 0);
sc.close();
}
```

Prueba de Escritorio

```
PS C:\Users\ready\OneDrive\Documents\PROGRAMACION_UTA\practicas>
=== SISTEMA DE TURNOS (LinkedList) ===
1.Agregar 2.Atender 3.Ver 0.Salir
-> 1
Nombre del paciente: Alex
Turno asignado a Alex

=== SISTEMA DE TURNOS (LinkedList) ===
1.Agregar 2.Atender 3.Ver 0.Salir
-> 2
Atendiendo a: Alex

=== SISTEMA DE TURNOS (LinkedList) ===
1.Agregar 2.Atender 3.Ver 0.Salir
-> 3
Turnos en espera:

=== SISTEMA DE TURNOS (LinkedList) ===
1.Agregar 2.Atender 3.Ver 0.Salir
-> 1
Nombre del paciente: Victor
Turno asignado a Victor

=== SISTEMA DE TURNOS (LinkedList) ===
1.Agregar 2.Atender 3.Ver 0.Salir
-> 0
PS C:\Users\ready\OneDrive\Documents\PROGRAMACION_UTA\practicas>
```

Imagen 31 y 32 Evidencia Java hecha en VisualCode

Pruebas de Escritorio

Caso	Entrada	Proceso	Salida Esperada
1	op=1: Ana; op=1: Luis; op=3	addLast x2; mostrar	- Ana - Luis
2	op=2 (atender)	removeFirst(); queda Luis	Atendiendo a: Ana
3	op=2 en lista vacía	isEmpty() = true	No hay turnos pendientes.



Ejercicio 10 – LinkedList – Simulación de Fila de Atención

Enunciado del Problema

Simular automáticamente la atención completa de una fila de clientes usando LinkedList/list. El programa registra n clientes ingresados por el usuario, luego los atiende uno por uno en orden FIFO, mostrando en cada paso quién es atendido, quién sigue y cuántos clientes quedan.

Análisis del Problema

A diferencia del ejercicio 9, este ejercicio atiende automáticamente a todos los clientes sin menú. Se utiliza un ciclo while que continúa mientras la lista no esté vacía, removiendo el primer elemento en cada iteración para simular la atención secuencial de una caja o ventanilla.

Variables de Entrada y Salida

Entrada

- n: Cantidad inicial de clientes en la fila (Entero).
- cliente: Variable de texto temporal.
- fila[n]: Arreglo dimensionado exactamente al tamaño de n.

Proceso

- Ejecutar un ciclo Para inicial para registrar todos los nombres de los clientes.
- Iniciar un puntero frente en 1.
- Ejecutar un ciclo Mientras que evalúa ininterrumpidamente si aún hay personas en la fila (frente <= n).
- Dentro del ciclo, simular la extracción FIFO (First In, First Out) aumentando la variable frente en 1.
- Realizar una operación aritmética simple (n - frente + 1) para determinar exactamente cuántas personas restan sin atender.

Salida

- Una traza automática en pantalla para cada iteración que muestra paso a paso:
 - "Atendiendo: [nombre]"
 - "Siguiente: [nombre en frente+1]" (si aplica)
 - "Restantes: [cantidad calculada]"
- Mensaje final de "Atencion finalizada." cuando se rompe la condición del ciclo.

Variable	Tipo de Dato	Descripción	E / S
N	Int	Número inicial de clientes en la fila	Entrada
Cliente	String / string	Nombre de cada cliente	Entrada
Fila	LinkedList<String> / list<string>	Cola de clientes a atender	Proceso
Atención mostrada	Texto	Secuencia completa de atención	Salida

Algoritmo

1. Inicio
2. Solicitar la cantidad n de clientes.
3. Ciclo n veces: leer nombre del cliente y agregar con addLast/push_back.
4. Ciclo while mientras la fila no esté vacía:
5. Mostrar el nombre del primer cliente (atender).
6. Eliminar el primer cliente con removeFirst/pop_front.
7. Si quedan clientes, mostrar el siguiente en la fila.



8. Mostrar el conteo de clientes restantes.
9. Cuando la fila quede vacía, mostrar mensaje de finalización.
10. Fin

Pseudocódigo (PSeInt)

Algoritmo FilaAtencion

Definir fila, cliente Como Caracter

Definir n, i, frente Como Entero

Escribir Sin Saltar "Total de clientes en fila: "

Leer n

Dimension fila[n]

Para i <- 1 Hasta n Hacer

 Escribir Sin Saltar "Cliente ", i, ": "

 Leer cliente

 fila[i] <- cliente

FinPara

frente <- 1

Mientras frente <= n Hacer

 Escribir "Atendiendo: ", fila[frente]

 frente <- frente + 1

 Si frente <= n Entonces

 Escribir "Siguiente: ", fila[frente]

 FinSi

 Escribir "Restantes: ", n - frente + 1

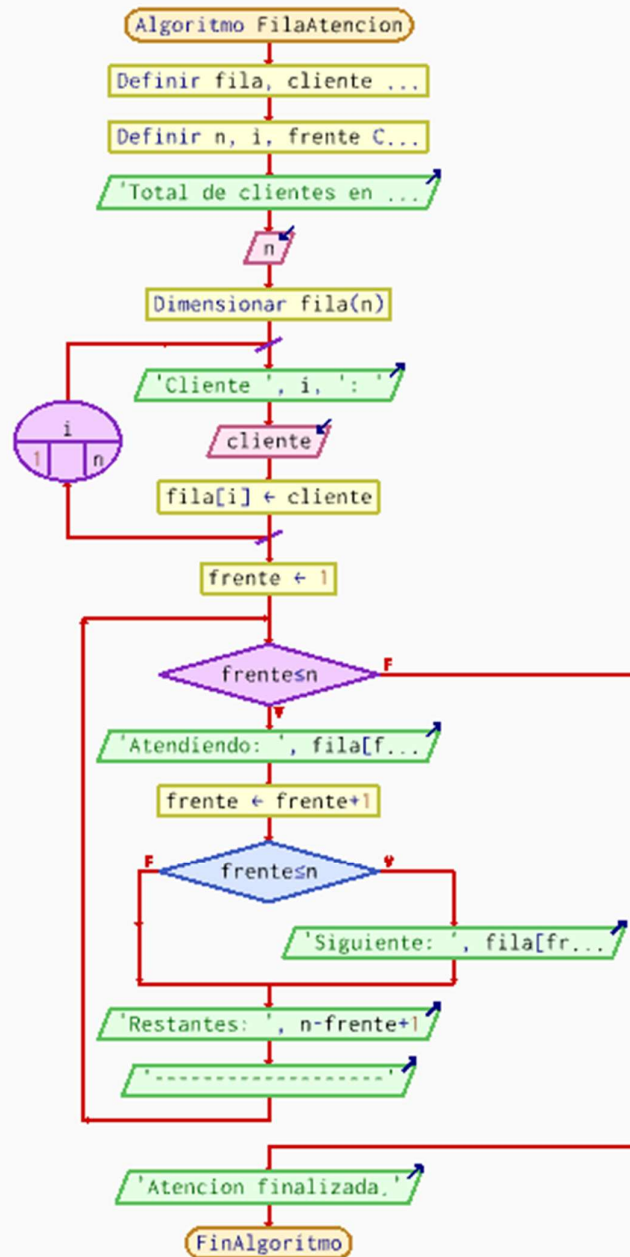
 Escribir "-----"

FinMientras

Escribir "Atencion finalizada."

FinAlgoritmo

Diagrama de Flujo



Código C++

```
#include <iostream>
#include <list>
#include <string>
using namespace std;
```

```
int main() {
    list<string> fila;
    int n; string cliente;
```

```
    cout << "Fila de clientes usando LinkedList/list" << endl;
    cout << "¿Cuántos clientes en la fila inicial? "; cin >> n;
    for (int i = 0; i < n; i++) {
```



```
        cout << "Cliente " << i+1 << ": "; cin >> cliente;
        fila.push_back(cliente);
    }

    cout << "\n--- SIMULACION DE ATENCION ---" << endl;
    while (!fila.empty()) {
        cout << "Atendiendo a: " << fila.front() << endl;
        fila.pop_front();
        if (!fila.empty())
            cout << "Siguiendo en fila: " << fila.front() << endl;
        cout << "Clientes restantes: " << fila.size() << endl;
    }
    cout << "\nFila de atencion finalizada." << endl;
    return 0;
}
```

Prueba de Escritorio

```
C:\msys64\ucrt64\bin\gdb.exe' '--interpreter=mi'
Fila de clientes usando LinkedList/list
¿Cuántos clientes en la fila inicial? 3
Cliente 1: Alex
Cliente 2: Jesenia
Cliente 3: Karen

--- SIMULACION DE ATENCION ---
Atendiendo a: Alex
Siguiendo en fila: Jesenia
Clientes restantes: 2
Atendiendo a: Jesenia
Siguiendo en fila: Karen
Clientes restantes: 1
Atendiendo a: Karen
Clientes restantes: 0

Fila de atencion finalizada.
PS C:\Users\ready\OneDrive\Documents\PROGRAMACION_UTA\practicas
>
```

Imagen 33 y 34 Evidencia C++ hecha en VisualCode

Código Java

```
import java.util.LinkedList;
import java.util.Scanner;

public class Ejercicio10 {
    public static void main(String[] args) {
        LinkedList<String> fila = new LinkedList<>();
        Scanner sc = new Scanner(System.in);

        System.out.print("¿Cuántos clientes en la fila inicial? ");
        int n = sc.nextInt();
        for (int i = 0; i < n; i++) {
            System.out.print("Cliente " + (i+1) + ": ");
            fila.add(sc.next());
        }

        System.out.println("\n--- SIMULACION DE ATENCION ---");
        while (!fila.isEmpty()) {
            System.out.println("Atendiendo a: " + fila.removeFirst());
            if (!fila.isEmpty())
                System.out.println("Siguiendo: " + fila.getFirst());
            System.out.println("Clientes restantes: " + fila.size());
        }
        System.out.println("\nFila de atencion finalizada.");
    }
}
```



```
sc.close();  
}  
}
```

Prueba de Escritorio

```
83541256\redhat.java\jdt_ws\practicas_d5ab8d49\bin' 'Ejercicio10'  
¿Cuántos clientes en la fila inicial? 3  
Cliente 1: Karla  
Cliente 2: Karen  
Cliente 3: Monserrath  
  
--- SIMULACION DE ATENCION ---  
Atendiendo a: Karla  
Siguiente: Karen  
Clientes restantes: 2  
Atendiendo a: Karen  
Siguiente: Monserrath  
Clientes restantes: 1  
Atendiendo a: Monserrath  
Clientes restantes: 0  
  
Fila de atencion finalizada.  
PS C:\Users\ready\OneDrive\Documents\PROGRAMACION_UTA\practicas>
```

```
¿Cuántos clientes en la fila inicial? 5  
Cliente 1: Samanta  
Cliente 2: Doménica  
Cliente 3: Solís  
Cliente 4: Selene  
Cliente 5: Estefanía  
  
--- SIMULACION DE ATENCION ---  
Atendiendo a: Samanta  
Siguiente: Doménica  
Clientes restantes: 4  
Atendiendo a: Doménica  
Siguiente: Solís  
Clientes restantes: 3  
Atendiendo a: Solís  
Siguiente: Selene  
Clientes restantes: 2  
Atendiendo a: Selene  
Siguiente: Estefanía  
Clientes restantes: 1  
Atendiendo a: Estefanía  
Clientes restantes: 0  
  
Fila de atencion finalizada.  
PS C:\Users\ready\OneDrive\Documents\PROGRAMACION_UTA\practicas>
```

Imagen 35 y 36 Evidencia C++ hecha en VisualCode

Pruebas de Escritorio

Caso	Entrada	Proceso	Salida Esperada
1	n=3: Rosa, Pedro, Ana	removeFirst x3, FIFO	Rosa→Pedro→Ana; Atencion finalizada.
2	n=1: Carlos	removeFirst x1	Atendiendo: Carlos; 0 restantes; Finalizado.
3	n=4: A,B,C,D	removeFirst x4 secuencial	A→B→C→D; cada paso muestra siguiente y restantes



2.8 Habilidades Blandas Empleadas

☒ Pensamiento crítico ☒ Adaptabilidad ☒ Responsabilidad ☐ Liderazgo ☐ Trabajo en equipo

2.9 Conclusiones

- La implementación de ArrayList (Java) y vector (C++) demostró ser altamente flexible para gestionar colecciones de datos sin tamaño fijo, permitiendo agregar y eliminar elementos en tiempo de ejecución sin redefinir estructuras, lo cual simplifica enormemente el desarrollo de aplicaciones dinámicas.
- Las LinkedList/list demostraron ser la estructura óptima para simular colas FIFO en sistemas de turnos y filas de atención, ya que las operaciones de inserción al final y eliminación al inicio se realizan en tiempo constante $O(1)$, sin desplazamiento de elementos.
- La comparación entre arreglos estáticos y listas dinámicas evidenció que, si bien los arreglos son más rápidos en acceso directo por índice, carecen de flexibilidad; mientras que ArrayList/vector ofrecen un balance óptimo entre rendimiento y funcionalidad para la mayoría de aplicaciones prácticas.
- El desarrollo paralelo en Java y C++ reforzó la comprensión de las equivalencias entre ambos lenguajes (ArrayList↔vector, LinkedList↔list, add↔push_back), consolidando una visión comparativa que facilita la transición entre lenguajes en contextos profesionales.

2.10 Recomendaciones

- Se recomienda utilizar ArrayList/vector cuando se necesite acceso aleatorio frecuente por índice, y LinkedList/list cuando las operaciones principales sean inserciones y eliminaciones al inicio o final de la colección, ya que cada estructura está optimizada para distintos patrones de uso.
- Es buena práctica validar que la lista no esté vacía antes de ejecutar operaciones como get(0), removeFirst() o pop_front(), para evitar excepciones de tiempo de ejecución (IndexOutOfBoundsException en Java o comportamiento indefinido en C++).
- Para proyectos con grandes volúmenes de datos, se sugiere inicializar el ArrayList/vector con una capacidad estimada (new ArrayList<>(capacidad) / vector.reserve(n)) para evitar múltiples reasignaciones de memoria que afectan el rendimiento.
- Mantener los proyectos correctamente versionados en GitHub con commits descriptivos facilita el seguimiento del progreso, la colaboración y la recuperación de versiones anteriores del código.

2.11 Referencias Bibliográficas

- [1] H. M. Deitel y P. J. Deitel, Cómo programar en C++, 9na ed. México: Pearson Educación, 2014.
- [2] J. Gosling, B. Joy, G. Steele, y G. Bracha, The Java Language Specification, Java SE 8 ed. Boston: Addison-Wesley, 2014.
- [3] B. Stroustrup, The C++ Programming Language, 4ta ed. Boston: Addison-Wesley, 2013.
- [4] Oracle, Java Platform SE 8 – ArrayList Documentation. [En línea]. Disponible: <https://docs.oracle.com/javase/8/docs/api/java/util/ArrayList.html>

2.12 Anexos

<https://github.com/AlexanderrrG/SW-AyLP-APE-06-Arreglos-y-Listas>